DAY 03 · DICTS & SETS

Dictionaries & Sets

Counting, grouping, sorting by value, and the traps hiding in mutable keys.

2–3 hours / day03_dicts_sets / Maps and sets

INSIDE THIS BOOKLET

Counting Grouping Sorting Sets

How to use this booklet

Each day is a working session, not a reading assignment. This booklet is the typeset version of the day's `README.md`: keep the folder open, type the examples into a Python REPL, and run the tests as you go. The *Exam lens* and end-of-day review are study aids added to the original material.

Read & type
Type every example. Predict the output first.

Practice
Implement or debug one test at a time.

Test
Run the suite. Read the first failing line.

Log
Record each bug in `bug_journal.md`.

You will need: Python 3.10+ · a terminal · `day03_dicts_sets/` · the course runner `python run.py 3`

In this session

Why dictionaries are everywhere 3

1 The basic operations 3

2 Looping over a dict, and the order you get 5

3 Counting and grouping 6

4 Sorting by value, and breaking ties 8

5 What can be a key? 9

6 Building dicts in one line: comprehensions 9

7 Dicts inside dicts, and copying 10

8 Sets 11

9 The toolbox: techniques the exercises rely on 12

Exercises 15

End-of-day checklist 16

> **◎ Today's goal**
>
> Dictionaries are the workhorse of almost every algorithm you'll debug (counts, indexes, vocabularies, caches, graphs). Master their API, iteration order, and the sorting/tie-break idioms.

Why dictionaries are everywhere

A list answers questions like "what's at position 3?". Very often, though, the question you actually have is "what do I know about *this thing*?": How many times have I seen the word `"cat"`? Which id did I give to the token `"th"`? Who are the neighbours of the city `"Lyon"`? Have I already computed the answer for `n = 40`? A **dictionary** (`dict`) answers exactly that kind of question. It stores pairs: a **key** (the thing you look up) and a **value** (what you've stored about it), just as a real dictionary pairs a word with its definition. Looking up a key is extremely fast, and it takes the same time whether the dictionary holds ten entries or ten million. How can it be that fast? Internally, Python turns each key into a number called its **hash**, and uses that number to jump straight to the place where the value is stored, instead of searching. You don't need the details. You need just one consequence of it, which comes up in section 5: keys must be values that never change. A **set** is a close relative: a collection of unique values with no associated data. It answers just one question, "is this in here?", and answers it just as fast. Almost every algorithm in this course leans on a dict: word counts, the tokenizer's vocabulary (Day 9), the key-value store (Day 10), the graphs in the drills, and the cache in drill 6. So today is worth doing carefully.

1 The basic operations

1.1 Creating, reading and writing

A dict is written with curly braces, with `key: value` pairs separated by commas:

```
>>> ages = {"ana": 31, "ben": 25}
>>> ages["ana"]              # read the value stored under "ana"
31
>>> ages["cai"] = 40         # add a new key...
>>> ages["ana"] = 32         # ...or overwrite an existing one: same syntax
>>> ages
{'ana': 32, 'ben': 25, 'cai': 40}
>>> len(ages)                # number of keys
3
```

There is no separate "insert" and "update": `d[key] = value` creates the key if it's new and replaces the value if it already exists.

1.2 Reading a key that isn't there

This is where most dictionary bugs live, so it's worth slowing down. There are two ways to read a value, and they behave differently when the key is **missing**. Square brackets raise an error:

```
>>> ages["zoe"]
KeyError: 'zoe'
```

`.get()` returns `None` instead, or a default value that you choose:

```
>>> print(ages.get("zoe"))
None
>>> ages.get("zoe", 0)
0
>>> ages.get("ana", 0)       # the key exists, so the default is ignored
32
```

Which one is correct depends on what the spec says should happen. "Raises `KeyError` if the key is absent" means square brackets. "Returns 0 for unknown words" means `.get(word, 0)`. Tests check this behaviour exactly, so it's worth reading that part of the spec carefully.

1.3 Checking whether a key exists: `in`

```
>>> "ana" in ages
True
>>> "zoe" in ages
False
>>> 31 in ages               # `in` looks at KEYS only, not values
False
>>> 32 in ages.values()      # to search the values, ask explicitly
True
```

`key in d` is instant. `value in d.values()` has to look at every value one after another, which is fine for small dicts but slow for big ones.

1.4 Removing keys

```
>>> del ages["cai"]          # remove; KeyError if missing
>>> ages.pop("ben")          # remove AND give back the value; KeyError if missing
25
>>> print(ages.pop("zoe", None))   # with a default: no error if missing
None
```

`d.pop(key, None)` is the usual way to say "remove this key if it's there, and don't complain otherwise".

1.5 Merging two dicts

```
>>> a = {"x": 1, "y": 2}
>>> b = {"y": 20, "z": 30}
>>> {**a, **b}               # a NEW dict; when both have a key, the later one (b) wins
{'x': 1, 'y': 20, 'z': 30}
>>> a | b                    # the same thing, in newer Python
{'x': 1, 'y': 20, 'z': 30}
>>> a.update(b)              # changes a ITSELF (in place), returns None
>>> a
{'x': 1, 'y': 20, 'z': 30}
```

The difference between "makes a new dict" and "changes the existing dict" matters, as we'll see in section 7.

2 Looping over a dict, and the order you get

2.1 Three ways to loop

```
for k in d:              # the keys
for v in d.values():     # the values
for k, v in d.items():   # (key, value) pairs, unpacked into two variables
```

The third form is the one you'll use most. `d.items()` produces pairs like `('ana', 32)`, and `for k, v in ...` splits each pair into two variables.

2.2 Dicts remember insertion order

Insertion order is guaranteed

Since Python 3.7, dicts remember insertion order. Re-assigning an existing key does *not* move it; deleting and re-inserting does.

Let's see what that means:

```
>>> d = {}
>>> d["b"] = 1
>>> d["a"] = 2
>>> d["c"] = 3
>>> list(d)                  # keys come out in the order they were added
['b', 'a', 'c']
>>> d["b"] = 100             # updating "b" keeps it in first place
>>> list(d)
['b', 'a', 'c']
>>> del d["b"]; d["b"] = 100 # deleting and re-adding moves it to the end
>>> list(d)
['a', 'c', 'b']
```

This matters because many specs depend on order: "return the words in the order they first appear", or "on a tie, the one seen first wins". Looping over a dict gives you that first-seen order for free. Deleting and re-inserting to move a key to the end is also exactly how a "least recently used" cache can be built (drill 6). To get the **oldest** key without a loop, use `next(iter(d))`. `iter(d)` prepares to walk through the keys, and `next` takes the first one.

2.3 Don't add or remove keys while looping

Don't mutate while iterating

Don't add/remove keys while iterating (`RuntimeError: dictionary changed size during iteration`). Iterate over a copy: `for k in list(d): ....`

```
prices = {"a": 0, "b": 5, "c": 0}
for k in prices:
    if prices[k] == 0:
        del prices[k]        # RuntimeError: dictionary changed size during iteration
```

The fix is to loop over a **snapshot** of the keys. `list(prices)` makes a separate list of the keys first, and deleting from the dict doesn't affect that list:

```python
for k in list(prices):
    if prices[k] == 0:
        del prices[k]        # fine
```

Changing the *value* of an existing key while looping is fine. Only adding or removing keys is a problem.

3 Counting and grouping

Two patterns cover a huge share of what dicts are used for in algorithm code. Learn to recognise both on sight.

3.1 Counting

Suppose you want to know how often each word appears in a list:

```python
words = ["the", "cat", "the", "hat", "the"]
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1
print(counts)        # {'the': 3, 'cat': 1, 'hat': 1}
```

The line `counts[w] = counts.get(w, 0) + 1` does three things: read the current count for `w` (or 0 if it's the first time we've seen `w`), add 1, and store the result back. Run through it by hand: on the first `"the"` the count goes from nothing to 1, on the second from 1 to 2, on the third from 2 to 3. A common planted bug is `counts.get(w, 1) + 1`, which counts every first occurrence as 2.

3.2 Grouping

Now suppose you want to group words by their length: `{3: ["the", "cat", ...], 5: [...]}`.

```python
groups = {}
for w in ["the", "cat", "horse", "zebra"]:
    groups.setdefault(len(w), []).append(w)
print(groups)        # {3: ['the', 'cat'], 5: ['horse', 'zebra']}
```

`setdefault(key, default)` means: "if `key` isn't in the dict yet, store `default` under it; either way, give me back the value that's stored there." So the first time we meet a 3-letter word, it stores an empty list under 3 and hands it back, and `.append(w)` adds the word to *that stored list*. The next 3-letter word gets the same list back and is added to it.

3.3 Ready-made helpers: `Counter` and `defaultdict`

The standard library module `collections` (allowed in the exam, since it's built into Python) has shortcuts for both patterns. `Counter` does the counting loop for you:

```
>>> from collections import Counter
>>> c = Counter("banana")
>>> c
Counter({'a': 3, 'n': 2, 'b': 1})
>>> c["a"]
3
>>> c["z"]                  # a missing key counts as 0, no error
0
>>> c.most_common(2)        # the 2 most frequent, highest first
[('a', 3), ('n', 2)]
```

When two items have the same count, `most_common` keeps them in the order they were first seen. `defaultdict` is a dict that automatically creates a value whenever you access a missing key. You tell it *how* to create that value: `defaultdict(list)` creates empty lists and `defaultdict(int)` creates zeros (because calling `int()` with no arguments gives 0).

```
>>> from collections import defaultdict
>>> groups = defaultdict(list)
>>> groups[3].append("cat")       # key 3 didn't exist: an empty list is created, then
    appended to
>>> groups[3].append("dog")
>>> dict(groups)
{3: ['cat', 'dog']}
>>> counts = defaultdict(int)
>>> counts["x"] += 1              # missing key starts at 0, then +1
>>> dict(counts)
{'x': 1}
```

3.4 Two traps

defaultdict inserts on read

Just by reading it (`dd["missing"]` inserts `[]`). Use `in` or `.get` to check existence.

```
>>> dd = defaultdict(list)
>>> if dd["maybe"]:          # looks like a harmless check...
...     pass
...
>>> dict(dd)                 # ...but it added the key
{'maybe': []}
```

If a test complains about unexpected extra keys, look for a `defaultdict` that's being *read* somewhere it should only be *checked*.

Shared list from fromkeys

ALL keys share the **same** list object. Use `{k: [] for k in keys}`.

`dict.fromkeys(keys, value)` creates a dict with every key mapped to `value`. With a number that's fine. But with `[]`, Python creates **one** empty list and stores that same list under every key. It's the same aliasing problem as the `[[0] * 3] * 3` grid from Day 1:

```
>>> index = dict.fromkeys(["a", "b"], [])
>>> index["a"].append(1)
>>> index
{'a': [1], 'b': [1]}
```

We appended to `"a"` and `"b"` changed too. The dict comprehension `{k: [] for k in keys}` (section 6) evaluates `[]` again for each key, so every key gets its own list.

4 Sorting by value, and breaking ties

4.1 The problem

Specs very often want a ranking: "the 3 most common words", "the highest-scoring player", "the most frequent pair". And almost always they add a rule for **ties**: "if two words have the same count, the alphabetically smaller one comes first". Getting the main order right but the tie rule wrong is a classic planted bug, because the code works on every input without ties.

4.2 Sorting the items of a dict

`sorted(counts.items(), key=...)` sorts the `(key, value)` pairs. The `key=` argument is a function that receives one pair and returns *what to sort by*. Here, `kv` is one pair, so `kv[0]` is the word and `kv[1]` is its count:

```
>>> counts = {"b": 3, "a": 3, "c": 5}
>>> sorted(counts.items(), key=lambda kv: kv[1], reverse=True)
[('c', 5), ('b', 3), ('a', 3)]
```

`c` comes first with 5. Then `b` and `a` are tied with 3, and they come out in their original dict order (`b` before `a`). That's because Python's sort is **stable**: items that compare equal keep their relative order. So if a spec says nothing about ties, "first seen wins" is what you get.

4.3 Putting the tie-break into the key

What if the spec says "by count, highest first; ties alphabetically"? Use a **tuple** as the sort key. Python compares tuples one element at a time: first elements first, and only if those are equal, the second elements.

```
>>> sorted(counts.items(), key=lambda kv: (-kv[1], kv[0]))
[('c', 5), ('a', 3), ('b', 3)]
```

The key for `("a", 3)` is `(-3, "a")`, for `("b", 3)` it's `(-3, "b")`, and for `("c", 5)` it's `(-5, "c")`. Sorting those keys in normal ascending order:

- `-5` is smaller than `-3`, so `c` comes first. Negating the count is what turns "highest count first" into ordinary ascending order.
- `a` and `b` both have `-3`, so the second element decides: `"a" < "b"`.

> **Tie-break in the key tuple**
>
> When a spec says "ties are broken by X", write the tie-break into the key tuple. `reverse=True` reverses **all** components of the key — so for "desc by count, asc by name" use `-count` rather than `reverse=True`.

In other words, `key=lambda kv: (kv[1], kv[0]), reverse=True` would give highest count first *and* reverse-alphabetical names on ties, which is not what the spec asked for.

4.4 `max` and `min` with a key

To get just the single best item, `max` and `min` accept the same `key=` argument:

```
>>> max(counts, key=counts.get)
'c'
```

This loops over the keys of `counts` and compares them using `counts.get(key)`, i.e. their counts. It returns the *key* with the largest count, not the count itself. When there's a tie, `max` and `min` return the **first** one they meet:

```
>>> max({"b": 3, "a": 3}, key={"b": 3, "a": 3}.get)
'b'
```

If the spec wants something else on ties, the key must say so. For example, "the pair with the highest count, and on ties the smallest pair" can be written as `min(pairs, key=lambda p: (-counts[p], p))`: the smallest `(-count, pair)` is the highest count, and among equal counts the smallest pair. The BPE tokenizer on Day 9 chooses which pair to merge with exactly this kind of line.

5 What can be a key?

Remember that a dict finds keys using their hash, a number computed from the key's contents. If a key could change after being stored, its hash would change too, and the dict would look for it in the wrong place. So Python only allows keys that can't change. Such values are called **hashable**. `str`, `int`, `float`, `bool`, `None`, `bytes`, `tuple` (of hashables), `frozenset` → OK. `list`, `dict`, `set` → `TypeError: unhashable type`. Use `tuple(lst)` as a key: `pair_counts[(a, b)] += 1` — this is exactly how BPE tokenizers count pairs.

```
>>> d = {}
>>> d[[1, 2]] = "x"
TypeError: unhashable type: 'list'
>>> d[(1, 2)] = "x"          # a tuple is fine
>>> d
{(1, 2): 'x'}
```

Tuples as keys are extremely common. A grid cell is `(row, col)`, a graph edge is `(from, to)`, two neighbouring tokens are `(a, b)`, and a dynamic-programming state is `(i, remaining_capacity)`. One curiosity: keys that are equal count as the *same* key, and Python considers `1`, `1.0` and `True` equal. Note `1 == 1.0 == True` so `{1: "a", True: "b"}` → `{1: 'b'}`. The second entry replaced the value of the first, because Python thinks they're the same key. You'll rarely hit this, but it explains odd results when booleans and numbers are mixed as keys.

6 Building dicts in one line: comprehensions

Just like list comprehensions (Day 1), a **dict comprehension** builds a dict from a loop. You write `key: value` followed by the loop:

```
>>> {x: x * x for x in range(5)}
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
>>> {w: len(w) for w in ["hi", "there"]}
{'hi': 2, 'there': 5}
```

A common use is **inverting** a dict, i.e. swapping keys and values. A tokenizer maps text to ids for encoding, and needs the reverse map from ids to text for decoding:

```
>>> token_to_id = {"a": 0, "b": 1}
>>> {v: k for k, v in token_to_id.items()}
{0: 'a', 1: 'b'}
```

This only works cleanly when every value is different. If two keys share a value, they both want to become the same new key, and the later one silently overwrites the earlier one:

```
>>> {v: k for k, v in {"x": 1, "y": 1}.items()}
{1: 'y'}
```

`"x"` has disappeared. When values can repeat, invert into lists instead: `inv.setdefault(v, []).append(k)`.

7 Dicts inside dicts, and copying

7.1 Reading nested data

Real data is often nested: dicts whose values are other dicts or lists.

```
>>> config = {"db": {"host": "x", "port": 5432}}
>>> config["db"]["port"]
5432
>>> config.get("cache", {}).get("size")    # safe even though "cache" doesn't exist
>>>
```

The second line uses `.get(..., {})`: if `"cache"` is missing, we get an empty dict, and calling `.get("size")` on an empty dict gives `None` instead of crashing.

7.2 Shallow and deep copies

Copying nested data is where bugs hide. There are two kinds of copy:

- A **shallow copy** (`dict(d)`, `d.copy()`) creates a new outer dict, but its values are the *same objects* as in the original. The inner dict is shared.
- A **deep copy** (`copy.deepcopy(d)`) copies everything, all the way down. Nothing is shared.

```
>>> import copy
>>> shallow = dict(config)
>>> shallow["db"]["port"] = 1
>>> config["db"]["port"]          # the original changed too!
1
>>> config["db"]["port"] = 5432
>>> deep = copy.deepcopy(config)
>>> deep["db"]["port"] = 1
>>> config["db"]["port"]          # untouched
5432
```

A shallow copy is enough when you'll only add, remove or replace top-level keys. If you'll modify anything *inside* the values, you need a deep copy.

7.3 The worst version: no copy at all

The most common bug in this area isn't a shallow copy, it's no copy at all:

> **Aliasing instead of copying**
>
> A function that "returns a new dict" but does `result = a` then mutates `result` mutates a — a classic bug.

```
def with_defaults(settings, defaults):
    """Return a NEW dict: defaults, overridden by settings. Don't modify either argument."""
    result = defaults                  # BUG: result and defaults are the same dict
    for key, value in settings.items():
        result[key] = value
    return result
```

As on Day 1, `result = defaults` doesn't copy anything. It attaches a second name to the same dict, so every change to `result` also changes the caller's `defaults`. The first call looks fine. But the next call starts from a `defaults` dict that still contains the previous caller's settings, so values leak from one call into the next. The fix is `result = dict(defaults)`. A shallow copy is enough here as long as the values are simple things like numbers and strings, which can't be modified in place.

8 Sets

8.1 What a set is for

A **set** holds unique values, in no particular order. You use it when the only question is membership: "have I already visited this node?", "is this word in the list of stop words?", "which items appear in both lists?".

```
>>> s = {1, 2, 3}
>>> s.add(4)                 # add a value (does nothing if it's already there)
>>> s.add(2)
>>> s
{1, 2, 3, 4}
>>> 3 in s                   # instant membership test
True
>>> set([1, 1, 2, 2, 3])     # converting to a set removes duplicates
{1, 2, 3}
```

An empty set must be written `set()`. Writing `{}` gives you an empty *dict*, for historical reasons. Removing values:

```
>>> s.discard(10)            # remove if present; silently does nothing otherwise
>>> s.remove(10)             # remove, but raise an error if it's not there
KeyError: 10
```

8.2 Why sets are faster than lists for membership

`x in some_list` checks the items one by one until it finds a match. For a list of 100,000 items, that's up to 100,000 comparisons. `x in some_set` uses hashing, like a dict, and is instant regardless of size. Inside a loop over many items (for example a graph search checking "have I seen this node?"), the difference is between a program that finishes instantly and one that times out.

8.3 Combining sets

Sets support the operations you may remember from maths:

```
>>> a = {1, 2, 3}
>>> b = {2, 3, 4}
>>> a | b        # union: in either
{1, 2, 3, 4}
>>> a & b        # intersection: in both
{2, 3}
>>> a - b        # difference: in a but not in b
{1}
>>> a ^ b        # symmetric difference: in exactly one of them
{1, 4}
>>> {2, 3} <= a  # subset: is every element of the left also in a?
True
```

8.4 Sets have no order

Sets have **no order** — never rely on iteration order of a set in output; sort it. The order in which a set gives you its elements is an internal detail. It can look sorted for small numbers and then change completely for strings, or between runs of the program. So a function that returns `list(some_set)` might pass a test today and fail tomorrow. If the output must be in a particular order, sort it (`sorted(some_set)`), or keep a separate list that records the order in which things were first seen.

9 The toolbox: techniques the exercises rely on

9.1 Chaining calls: `d.setdefault(k, []).append(x)`

You met `setdefault` in section 3. It's almost always used in a chain, which is worth reading slowly:

```
groups.setdefault(key, []).append(word)
```

Python evaluates this left to right:

1. `groups.setdefault(key, [])` makes sure `groups[key]` exists (creating an empty list if it didn't), and **returns the list that's stored there**.
2. `.append(word)` is then called on *that returned list*, which is the very list inside the dict.

So the word lands in the dict, without ever writing `groups[key]` twice. It works with any container that has a method for adding: with a set it becomes `adj.setdefault(a, set()).add(b)`. The equivalent long version, if you find it clearer:

```
if key not in groups:
    groups[key] = []
groups[key].append(word)
```

9.2 Looping over pairs: `for a, b in edges`

A list of pairs, like the edges of a graph `[("a", "b"), ("b", "c")]`, can be unpacked directly in the `for` line (Day 1, section 8). Each round, the pair is split into `a` and `b`. `for k, v in d.items()` (section 2) is the same idea.

9.3 Returning a pair, or `None`

A function can return a tuple: `return (i, j)`. The parentheses are optional (`return i, j` means the same), but they make it obvious that one value, a pair, comes back. When there's no answer, the convention is to return `None`, and the caller checks with `if result is None:`.

9.4 What kind of value is this? `isinstance`

With nested data, a value might be a plain number or string, or it might be another dict you have to go into. To ask which, use `isinstance(value, type)`:

```
>>> isinstance({"a": 1}, dict)
True
>>> isinstance(5, dict)
False
>>> isinstance(5, (int, float))        # a tuple of types means "any of these"
True
```

Typical use: `if isinstance(v, dict): ... go one level deeper ... else: ... it's a leaf value ...`. You'll also see the combination "is it a dict, **and** does it have this key?":

```
if not isinstance(cur, dict) or part not in cur:
    return default
```

The order matters. `or` stops as soon as it finds something true, so if `cur` isn't a dict, `part not in cur` is never evaluated. That's good, because `part not in 5` would raise a `TypeError`.

9.5 Walking down a dotted path

A path like `"db.primary.port"` is turned into a list of keys with `path.split(".")`, giving `["db", "primary", "port"]`. Then a loop goes one level deeper per key, keeping a variable that points at "where we are now":

```
cur = config                        # start at the top
for part in "db.primary.port".split("."):
    cur = cur[part]                 # step one level down
# cur is now config["db"]["primary"]["port"]
```

To **set** a value at a path, you have to stop one level early, because the last key is the one you assign to, not the one you step into. That's what `parts[:-1]` (all keys except the last) and `parts[-1]` (the last key) are for:

```
parts = path.split(".")
cur = d
for part in parts[:-1]:             # walk down to the dict that will hold the value
    ...                             # (creating missing levels with cur[part] = {} on the way)
    cur = cur[part]
cur[parts[-1]] = value              # assign in that dict
```

9.6 Recursion: a function that calls itself

Some data is nested to an unknown depth: a dict may contain a dict that contains a dict... A loop can't easily handle "as many levels as there happen to be", but a function that **calls itself** on each nested part can. That's **recursion**. The trick to writing (and reading) a recursive function is to stop thinking about the whole thing, and only answer two questions:

1. **The simple case.** What do I do with a value that isn't nested? (Usually: just use it.)
2. **The nested case.** If I *assume* the function already works correctly on the smaller, inner dict, how do I use its result?

Flattening is a good example. We want `{"a": 1, "b": {"c": 2, "d": {"e": 3}}}` to become `{"a": 1, "b.c": 2, "b.d.e": 3}`.

```
def flatten_dict(d, prefix=""):
    out = {}
    for k, v in d.items():
        key = f"{prefix}.{k}" if prefix else k     # "b" at the top, "b.c" one level down
        if isinstance(v, dict):
            out.update(flatten_dict(v, key))      # nested: flatten the inner dict, prefixed
                with our key
        else:
            out[key] = v                          # simple: store the value under the full key
    return out
```

Follow the calls on that example:

```
flatten_dict({"a": 1, "b": {...}}, "")
  "a" -> 1 is simple: out["a"] = 1
  "b" -> a dict: call flatten_dict({"c": 2, "d": {...}}, "b")
           "c" -> simple: out["b.c"] = 2
           "d" -> a dict: call flatten_dict({"e": 3}, "b.d")
                    "e" -> simple: out["b.d.e"] = 3
                    returns {"b.d.e": 3}
           returns {"b.c": 2, "b.d.e": 3}
  out.update(...) adds those keys
  returns {"a": 1, "b.c": 2, "b.d.e": 3}
```

Each call handles one level and hands the deeper levels to another call of itself. The calls stop when there are no more dicts inside, which is why every recursive function needs a simple case that doesn't recurse. `out.update(other)` (section 1) copies every key/value of `other` into `out`. Recursion is also a neat way to copy nested dicts. Calling a "merge" function with an empty second dict, as in `deep_merge(v, {})`, rebuilds `v` level by level, so the result shares no inner dicts with the input. That matters when the spec says "neither input may be modified".

9.7 Raising `KeyError` and `ValueError` yourself

Section 1 showed Python raising `KeyError` when a key is missing. Your own functions can raise it too, when the spec says "KeyError if the item doesn't exist": `raise KeyError(name)`. By convention, the message of a `KeyError` is just the missing key. `ValueError` is for "the key exists but the value doesn't make sense", like removing more units than are in stock.

9.8 `sorted` on a generator, with a key

`sorted` accepts anything you can loop over, including a generator expression (Day 1, section 8):

```
sorted((name for name, qty in inv.items() if qty < 5), key=lambda n: (inv[n], n))
```

Read it in two parts. The generator produces the names whose quantity is below 5. The key sorts them by quantity, and by name when quantities are equal (section 4). The result is a list of names.

Exercises

Practice lab

Work in `exercises/`. Read each docstring as the spec. Run `python run.py 3` after every change; peek at `solutions/` only after 15 minutes stuck.

File	Type	What
`exercises/ex01_counting.py`	implement	word frequencies, top-k with tie-breaks, first unique char
`exercises/ex02_grouping.py`	implement	anagram groups, invert dict, group_by, two-sum
`exercises/ex03_nested.py`	implement	get/set by dotted path, flatten/unflatten, deep merge
`exercises/ex04_fix_me.py`	**debug**	an inventory module with 6 bugs

Background for each:

- **ex01** is counting (section 3) plus ranking with tie-breaks (section 4). "Top-k" means "the k best". "First unique character" means the first character that appears exactly once: count everything first, then walk through the string again in order.
- **ex02** is grouping. Two words are **anagrams** if they use exactly the same letters, like "listen" and "silent". A neat trick: sort each word's letters (`"".join(sorted("listen"))` gives `"eilnst"`). All anagrams produce the same sorted string, so you can use it as the grouping key. **Two-sum** asks: given a list of numbers and a target, find two positions whose numbers add up to the target. The slow way tries every pair. The dict way walks through the list once and, for each number `x`, asks "have I already seen `target - x`?", using a dict from value to position. It's a classic example of a dict replacing a loop inside a loop.
- **ex03** works with nested data. A **dotted path** like `"db.port"` is a compact way of writing `config["db"]["port"]`. **Flattening** turns `{"db": {"port": 1}}` into `{"db.port": 1}`, and **unflattening** reverses it. These are natural candidates for recursion (a function that calls itself for each nested level), or for a loop that walks down one level at a time.
- **ex04** is an inventory, a dict from item name to quantity, with six functions that each contain one bug. Several of them are traps from this document.

Which tools each function needs (section numbers in brackets; "D1"/"D2" mean Day 1/Day 2):

Function	Tools
`word_frequencies`	`split()`, `strip(chars)`, `lower` (D2), `d.get(k, 0) + 1` (3)
`top_k_words`	`sorted(d.items(), key=lambda kv: (-kv[1], kv[0]))` (4), slicing `[:k]`
`first_unique_char`	counting (3), then `enumerate` over the string (D1)
`most_common_char`	counting (3), `max(d, key=d.get)` (4)

Function	Tools
`common_elements`	`set(a) & set(b)` (8), `sorted`
`group_anagrams`	`"".join(sorted(w))` as the key (D2), `setdefault(...).append` (9), `list(d.values())`
`invert_dict`	`for k, v in d.items()`, `setdefault` (9), a dict comprehension (6)
`group_by`	calling the function you're given, `key_fn(x)`, then `setdefault` (9)
`two_sum`	`enumerate`, a dict from value to index, `return (i, j)` / `None` (9)
`build_graph`	`for a, b in edges`, `setdefault(a, set()).add(b)` (9), dict comprehension with `sorted`
`get_path` / `set_path`	`split(".")`, walking down a path, `parts[:-1]`, `isinstance` (9)
`flatten_dict` / `deep_merge`	recursion, `isinstance`, `update` (9)
`unflatten_dict`	`set_path` in a loop
`ex04_fix_me`	`get`, `del`, `dict(a)` copies (7), `fromkeys` trap (3), `sum(d.values())`

```
python run.py 3
```

End-of-day checklist

Tick every box

- [ ] I can sort `(key, count)` pairs by count desc then key asc without thinking.
- [ ] I know what `max(d, key=d.get)` returns on ties.
- [ ] I know why `dict.fromkeys(k, [])` and `result = original` are bugs.

Exam lens

- Counting and grouping are the backbone of tokenizers, caches, and parsers; know the idioms cold.
- Write tie-breaks into the sort key. `reverse=True` reverses every component, not just the count.
- Keys must be hashable. Convert lists to tuples before counting pairs.

Tonight's bug journal

Where	Symptom	Root cause / fix

One row per bug. Re-read this page on Day 10.